

Трансформации и анимации

Докладчики: Волкова Анастасия, Немчанинов Руслан

1. CSS ТРАНСФОРМАЦИИ (transform)

Модель визуального форматирования CSS описывает систему координат внутри каждого позиционированного элемента. Система координат является точкой отсчета для свойств смещения. Положение и размеры в этом координатном пространстве можно рассматривать как заданные в пикселях, относительно точки отсчета, с положительными значениями, идущими вправо и вниз. Это координатное пространство можно изменить с помощью свойства `transform`.

CSS-трансформации позволяют сдвигать, поворачивать и масштабировать элементы. Трансформации преобразовывают элемент, не затрагивая остальные элементы веб-страницы, т.е. другие элементы не сдвигаются относительно него.

1. CSS ТРАНСФОРМАЦИИ (transform)

Трансформированным считается элемент с любым установленным значением свойства `transform`, отличным от `none`.

Существуют два вида CSS-трансформаций – **2D** и **3D**. **2D-трансформации** преобразовывают элементы в двумерном пространстве с помощью 2D-матрицы преобразований. Эта матрица применяется для вычисления новых координат объекта, на основе значений свойств `transform` и `transform-origin`. Преобразования влияют только на визуальный рендеринг. В отношении макета страницы они могут отразиться на переполнении содержимого блока. По умолчанию точка трансформации находится в центре элемента.

1.1. Функции 2D-трансформации: свойство `transform`

Свойство `transform` задает вид преобразования элемента. Свойство описывается с помощью **функций трансформации**, которые смещают элемент относительно его текущего положения на странице или изменяют его первоначальные размеры и форму.

Свойство не наследуется.

```
selector {  
  transform: none | <transform-function> [<transform-function>...];  
}
```

1.1. Функции 2D-трансформации: свойство transform

Функция	Описание
<code>none</code>	Значение по умолчанию, означает отсутствие трансформации. Также отменяет трансформацию для элемента из группы трансформируемых элементов.
<code>translate(x,y)</code>	Сдвигает элемент на новое место, перемещая относительно обычного положения вправо и вниз, используя координаты X и Y, не затрагивая при этом соседние элементы. Если нужно сдвинуть элемент влево или вверх, то нужно использовать отрицательные значения.

Функция `matrix(a, c, b, d, x, y)`

Объединяет ВСЕ 2D-трансформации в одной функции:

• Поворот • Масштабирование • Наклон • Перемещение

Параметр	Что делает
a	Масштаб по горизонтали (0-1=уменьшение, >1=увеличение)
b	Наклон по оси Y (+=вверх, -=вниз)
c	Наклон по оси X (+=влево, -=вправо)
d	Масштаб по вертикали (<1=уменьшение, >1=увеличение)
x	Смещение по оси X (+=вправо, -=влево)
y	Смещение по оси Y (+=вниз, -=вверх)

Пример: `matrix(1, 0.2, -0.2, 1, 50, 100)`

1.1. Функции 2D-трансформации: свойство transform

Функция	Описание
<code>translateX(n)</code>	Сдвигает элемент относительно его обычного положения по оси X.
<code>translateY(n)</code>	Сдвигает элемент относительно его обычного положения по оси Y.
<code>scale(x,y)</code>	Масштабирует элементы, делая их больше или меньше. Значения от 0 до 1 уменьшают элемент. Первое значение масштабирует элемент по ширине, второе — по высоте. Отрицательные значения отображают элемент зеркально.
<code>scaleX(n)</code>	Функция масштабирует элемент по ширине, делая его шире или уже. Если значение больше единицы, элемент становится шире, если значение находится между единицей и нулем, элемент становится уже. Отрицательные значения отображают элемент зеркально по горизонтали.

Положительные значения сдвигают вправо/вниз, отрицательные — влево/вверх .

```
/* Пример использования */  
  
.moved-element {  
  
  transform: translate(50px, 100px);  
  
  /* Элемент визуально сместится, но место в макете сохранится */  
  
}
```

Функция	Описание
<code>scaleY(n)</code>	Функция масштабирует элемент по высоте, делая его выше или ниже. Если значение больше единицы, элемент становится выше, если значение находится между единицей и нулем — ниже. Отрицательные значения отображают элемент зеркально по вертикали.
<code>rotate(угол)</code>	Поворачивает элементы на заданное количество градусов, отрицательные значения от -1deg до -360deg поворачивают элемент против часовой стрелки, положительные — по часовой стрелке. Значение <code>rotate(720deg)</code> поворачивает элемент на два полных оборота.
<code>skew(x- угол, y-угол)</code>	Используется для деформирования (искажения) сторон элемента относительно координатных осей. Если указано одно значение, второе будет определено браузером автоматически.
<code>skewX(угол)</code>	Деформирует стороны элемента относительно оси X.
<code>skewY(угол)</code>	Деформирует стороны элемента относительно оси Y.
<code>initial</code>	Устанавливает значение свойства в значение по умолчанию.
<code>inherit</code>	Наследует значение свойства от родительского элемента.

- **1** — исходный размер (100%)
- **> 1** — увеличение (1.5 = 150%)
- **< 1** — уменьшение (0.5 = 50%)

Отрицательные значения —
отражение элемента

```
/* Увеличение в 1.5 раза */
```

```
.enlarged {  
  transform: scale(1.5);  
}
```

```
/* Отражение по горизонтали */
```

```
.mirrored {  
  transform: scaleX(-1);  
}
```

Единицы измерения угла:

- deg — градусы ($360\text{deg} = \text{полный круг}$)
- rad — радианы ($2\pi\text{rad} = 360^\circ$)
- turn — обороты ($1\text{turn} = 360^\circ$)
- grad — грады ($400\text{grad} = 360^\circ$)

```
/* Поворот на 45 градусов по часовой  
стрелке */
```

```
.rotated {  
  transform: rotate(45deg);  
}
```

```
/* Полный оборот */
```

```
.full-circle {  
  transform: rotate(1turn);  
}
```

`transform: none;`

`transform: matrix(1.0, 2.0, 3.0, 4.0, 5.0, 6.0);`

`transform: rotate(45deg);`

`transform: translate(12px, 50%);`

`transform: translateX(2em);`

`transform: translateY(3in);`

`transform: scale(2, 0.5);`

`transform: scaleX(2);`

`transform: scaleY(0.5);`

`transform: skew(30deg, 20deg);`

`transform: skewX(30deg);`

`transform: skewY(1.07rad);`

`transform: translateX(10px) rotate(10deg) translateY(5px);`

`transform: inherit;`

`transform: initial;`

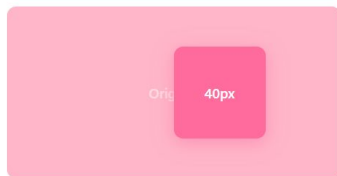
translate(x, y)



```
transform: translate(30px, 20px);
```

Сдвиг по X и Y осям

translateX(n)



```
transform: translateX(40px);
```

Только по горизонтали

translateY(n)



```
transform: translateY(-30px);
```

Только по вертикали

scale(x, y)



```
transform: scale(1.5, 0.8);
```

Масштаб по ширине и высоте

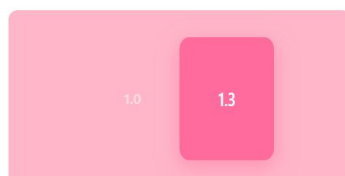
scaleX(n)



```
transform: scaleX(1.5);
```

Только по ширине

scaleY(n)



```
transform: scaleY(1.3);
```

Только по высоте

rotate(угол)



```
transform: rotate(45deg);
```

Поворот по часовой стрелке

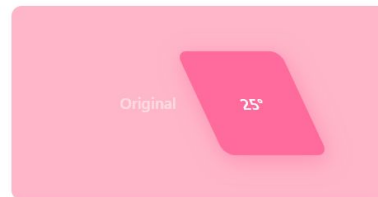
skew(x, y)



```
transform: skew(20deg, 10deg);
```

Наклон по обеим осям

skewX(угол)



```
transform: skewX(25deg);
```

Наклон по оси X

skewY(угол)



```
transform: skewY(20deg);
```

Наклон по оси Y

1.2. Точка трансформации: свойство `transform-origin`

Свойство `transform-origin` позволяет сместить центр трансформации, относительно которого происходит изменение положения/размера/формы элемента. Значение по умолчанию — center, или 50% 50%. Задается только для трансформированных элементов.

Свойство не наследуется.

1.2. Точка трансформации: свойство transform-origin

transform-origin	
Значения:	Описание
ось X(left, center, right, длина, %) ось Y(top, center, bottom, длина, %)	Пара значений, заданная с помощью ключевых слов, единиц длины или процентов определяет, относительно какой части элемента будет происходить трансформация. Значения больше 100% увеличивают область трансформации элемента.
initial	Устанавливает значение свойства в значение по умолчанию.
inherit	Наследует значение свойства от родительского элемента.

`transform-origin: 2px;`

`transform-origin: bottom;`

`transform-origin: 3cm 2px;`

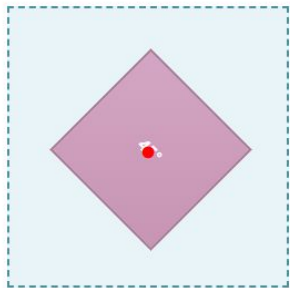
`transform-origin: left 2px;`

`transform-origin: right top;`

`transform-origin: inherit;`

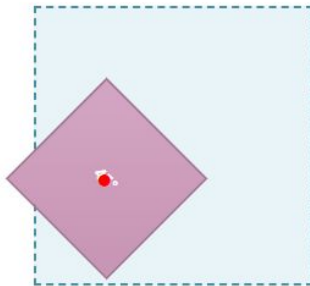
`transform-origin: initial;`

`transform-origin: center center`



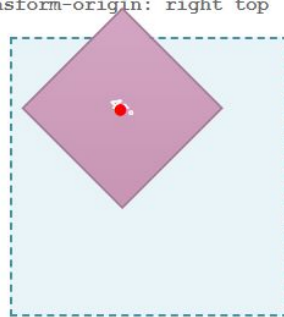
По умолчанию

`transform-origin: left 2px`



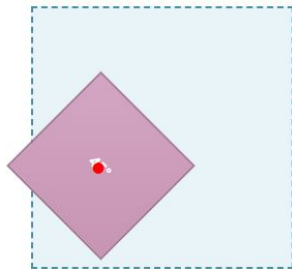
Ключевое слово + длина

`transform-origin: right top`



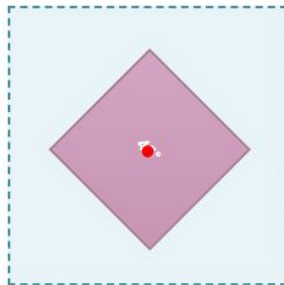
Правый верхний угол

`transform-origin: 2px 2px`



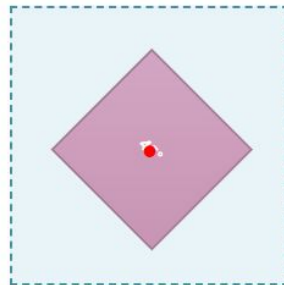
В пикселях

`transform-origin: inherit`



Наследуется от родителя

`transform-origin: initial`



Значение по умолчанию

1.3. Область преобразования: свойство `transform-box`

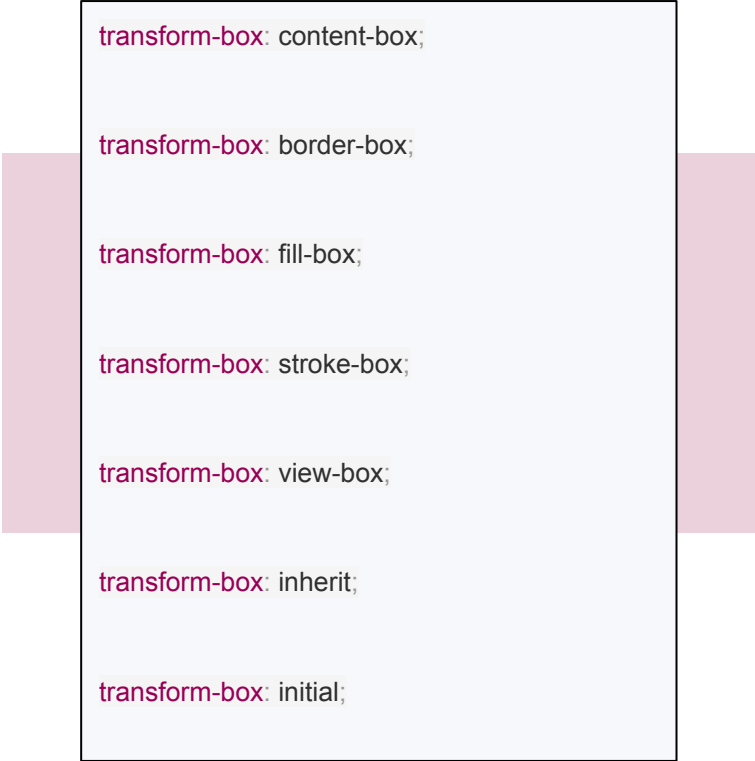
Все преобразования, определяемые свойством `transform` и `transform-origin`, относятся к положению и размерам опорного блока элемента. Опорный блок элемента это виртуальный прямоугольник вокруг элемента, который формирует систему координат для отрисовки.

Опорный блок добавляет дополнительное смещение к исходной точке, заданной свойством `transform-origin`.

Свойство не наследуется.

1.3. transform-box

Значения:	Описание
content-box	В качестве опорного блока выступает область содержимого элемента. Для элемента <table> эта область включает также заголовок таблицы и рамку.
border-box	В качестве опорного блока выступает область рамки элемента. Для элемента <table> эта область включает также заголовок таблицы и рамку.
fill-box	В качестве опорного блока выступает ограничивающая рамка, содержащая только геометрическую форму элемента.
stroke-box	В качестве опорного блока выступает ограничивающая рамка, содержащая геометрическую форму и обводку элемента.
view-box	В качестве опорного блока используется область просмотра на SVG-холсте, которая определяет прямоугольную область, в которую отображается содержимое SVG.
initial	Устанавливает значение свойства в значение по умолчанию.
inherit	Наследует значение свойства от родительского элемента.



`transform-box: content-box;`

`transform-box: border-box;`

`transform-box: fill-box;`

`transform-box: stroke-box;`

`transform-box: view-box;`

`transform-box: inherit;`

`transform-box: initial;`

1.4. Комбинирование функций

Несколько функций можно комбинировать в одном правиле . Порядок важен — функции применяются слева направо.

```
/* Сначала перемещение, потом поворот, потом масштаб */  
.combined {  
  transform: translateX(50px) rotate(30deg) scale(1.2);  
}
```

2. CSS-переходы

CSS-переходы позволяют анимировать исходное значение CSS-свойства на новое значение с течением времени, управляя скоростью смены значений свойств. Большинство свойств меняют свои значения за 16 миллисекунд, поэтому рекомендуемое время стандартного перехода — 200ms.

Смена свойств происходит при наступлении определенного события, которое описывается соответствующим псевдоклассом. Чаще всего используется псевдокласс `:hover`. Данный псевдокласс не работает на мобильных устройствах, таких как iPhone или Android. Универсальным решением, работающим в настольных и мобильных браузерах, будет обработка событий с помощью JavaScript (например, переключение классов при клике).

Переходы применяются ко всем элементам, а также к псевдоэлементам `:before` и `:after`. Для задания всех свойств перехода обычно используют краткую запись свойства `transition`.

CSS3-переходы могут применяться не ко всем свойствам и их значениям.

2.1. Название свойства transition-property

Содержит название CSS-свойств, к которым будет применен эффект перехода. Значение свойства может содержать как одно свойство, так и список свойств через запятую. При создании перехода можно использовать как начальное, так и конечное состояние элемента. Свойство не наследуется.

```
div {  
  width: 100px;  
  transition-property: width;  
}  
div:hover {  
  width: 300px;  
}
```

2.1. Название свойства transition-property

transition-property	
Значения:	Описание
none	Отсутствие свойства для перехода.
all	Значение по умолчанию. Применяет эффект перехода ко всем свойствам элемента.
свойство	Определяет список CSS-свойств, перечисленных через запятую, участвующих в переходе.
initial	Устанавливает значение свойства в значение по умолчанию.
inherit	Наследует значение свойства от родительского элемента.

`/* Полная запись */`

```
.element {  
  transition-property: transform;  
  transition-duration: 0.3s;  
  transition-timing-function:  
ease-in-out;  
  transition-delay: 0.1s;  
}
```

`/* Краткая запись */`

```
.element {  
  transition: transform 0.3s  
ease-in-out 0.1s;  
}
```

`/* Несколько свойств */`

```
.element {  
  transition: transform 0.3s ease,  
opacity 0.5s linear;  
}
```

2.2. Продолжительность перехода

transition-duration

Задаёт промежуток времени, в течение которого должен осуществляться переход. Если разные свойства имеют разные значения для перехода, они указываются через запятую. Если продолжительность перехода не указана, то анимация при смене значений свойств происходить не будет. Свойство не наследуется.

```
div {  
  
  transition-duration: .2s;  
  
}
```

2.2. Продолжительность перехода

transition-duration

transition-duration	
Значения:	Описание
время	Время перехода указывается в секундах или миллисекундах, например, 2s или 5ms.
initial	Устанавливает значение свойства в значение по умолчанию.
inherit	Наследует значение свойства от родительского элемента.

2.3. Функция перехода

transition-timing-function

Свойство задает временную функцию, которая описывает скорость перехода объекта от одного значения к другому. Если вы определяете более одного перехода для элемент, например, цвет фона элемента и его положение, вы можете использовать разные функции для каждого свойства. Свойство не наследуется.

```
div {  
  
  transition-timing-function: linear;  
  
}
```

transition-timing-function

Значения:	Описание
ease	Функция по умолчанию, переход начинается медленно, разгоняется быстро и замедляется в конце. Соответствует cubic-bezier(0.25,0.1,0.25,1).
linear	Переход происходит равномерно на протяжении всего времени, без колебаний в скорости. Соответствует cubic-bezier(0,0,1,1).
ease-in	Переход начинается медленно, а затем плавно ускоряется в конце. Соответствует cubic-bezier(0.42,0,1,1).
ease-out	Переход начинается быстро и плавно замедляется в конце. Соответствует cubic-bezier(0,0,0.58,1).
ease-in-out	Переход медленно начинается и медленно заканчивается. Соответствует cubic-bezier(0.42,0,0.58,1).
cubic-bezier(x1, y1, x2, y2)	Позволяет вручную установить значения от 0 до 1 для кривой ускорения. На этом сайте вы сможете построить любую траекторию перехода.
initial	Устанавливает значение свойства в значение по умолчанию.
inherit	Наследует значение свойства от родительского элемента.

```
/* Разные функции времени */
```

```
.linear-transition {  
  transition: all 1s linear;  
}
```

```
.ease-transition {  
  transition: all 1s ease;  
}
```

```
.custom-transition {  
  transition: all 1s cubic-bezier(0.68, -0.55, 0.265,  
  1.55);  
}
```

```
.steps-transition {  
  transition: all 1s steps(5);  
}
```

2.4. Задержка перехода transition-delay

Необязательное свойство, позволяет сделать так, чтобы изменение свойства происходило не моментально, а с некоторой задержкой. Не наследуется.

```
div {  
  
  transition-delay: .5s;  
  
}
```

2.4. Задержка перехода transition-delay

transition-delay	
Значения:	Описание
время	Время задержки перехода указывается в секундах или миллисекундах.
initial	Устанавливает значение свойства в значение по умолчанию.
inherit	Наследует значение свойства от родительского элемента.

2.5. Краткая запись перехода

Все свойства, отвечающие за изменение внешнего вида элемента, можно объединить в одно свойство `transition`

`transition: transition-property transition-duration transition-timing-function transition-delay;`

Если воспользоваться значениями по умолчанию, то запись

будет эквивалентна

```
div {transition: 1s;}
```

```
div {transition: all 1s ease 0s;}
```

2.6. Плавный переход нескольких свойств

Для элемента можно задать несколько последовательных переходов, перечислив их через запятую. Каждый переход можно оформить своей временной функцией.

```
div {transition: background 0.3s ease, color 0.2s linear;}
```

или

```
div {  
  transition-property: height, width, background-color;  
  transition-duration: 3s;  
  transition-timing-function: ease-in, ease, linear;  
}
```

3. CSS АНИМАЦИИ

CSS-анимация придаёт сайтам динамичность. Она оживляет веб-страницы, улучшая взаимодействие с пользователем. В отличие от CSS-переходов, создание анимации базируется на ключевых кадрах, которые позволяют автоматически воспроизводить и повторять эффекты на протяжении заданного времени, а также останавливать анимацию внутри цикла.

CSS-анимация может применяться практически для всех html-элементов, а также для псевдоэлементов `:before` и `:after`.

3.1. Ключевые кадры

Ключевые кадры используются для указания значений свойств анимации в различных точках анимации. Ключевые кадры определяют поведение одного цикла анимации; анимация может повторяться ноль или более раз.

Ключевые кадры указываются с помощью правила `@keyframes`, определяемого следующим образом:

```
@keyframes имя анимации { список правил }
```

Создание анимации начинается с установки **ключевых кадров** правила `@keyframes`. Кадры определяют, какие свойства на каком шаге будут анимированы. Каждый кадр может включать один или более блоков объявления из одного или более пар свойств и значений. Правило `@keyframes` содержит имя анимации элемента, которое связывает правило и блок объявления элемента.

3.1. Ключевые кадры

```
@keyframes shadow {  
  from {text-shadow: 0 0 3px black;}  
  50% {text-shadow: 0 0 30px black;}  
  to {text-shadow: 0 0 3px black;}  
}
```

Ключевые кадры создаются с помощью ключевых слов `from` и `to` (эквивалентны значениям `0%` и `100%`) или с помощью процентных пунктов, которых можно задавать сколько угодно. Также можно комбинировать ключевые слова и процентные пункты. Если кадры имеют одинаковые свойства и значения, их можно объединить в одно объявление:

3.1. Ключевые кадры

Если 0% или 100% кадры не указаны, то браузер пользователя создает их, используя вычисляемые (первоначально заданные) значения анимируемого свойства.

Если несколько правил `@keyframes` определены с одним и тем же именем, сработает последнее в порядке документа, а все предыдущие проигнорируются.

```
@keyframes move {  
  from,  
  to {  
    top: 0;  
    left: 0;  
  }  
  25%,  
  75% {top: 100%;}  
  50% {top: 50%;}  
}
```

3.1. Ключевые кадры

После объявления правила `@keyframes`, мы можем сослаться на него в свойстве `animation`:

```
h1 {  
  font-size: 3.5em;  
  color: darkmagenta;  
  animation: shadow 2s infinite ease-in-out;  
}
```

3.1. Временная функция для ключевых кадров

Правило стиля ключевого кадра также может объявлять временную функцию, которая должна использоваться при перемещении анимации к следующему ключевому кадру.

```

/ @keyframes bounce {
  from {
    top: 100px;
    animation-timing-function: ease-out;
  }
  25% {
    top: 50px;
    animation-timing-function: ease-in;
  }
  50% {
    top: 100px;
    animation-timing-function: ease-out;
  }
  75% {
    top: 75px;
    animation-timing-function: ease-in;
  }
  to {
    top: 100px;
  }
}

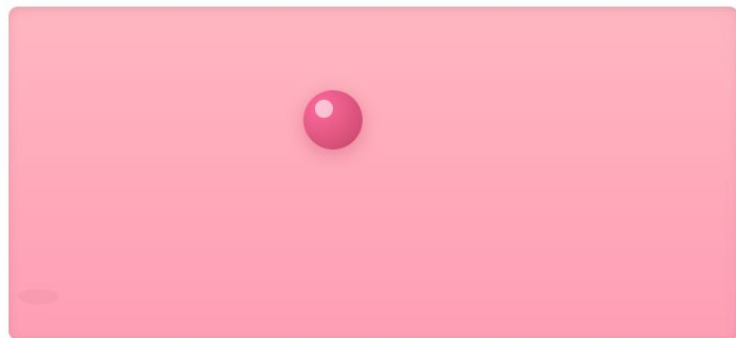
```

```
@keyframes fadeIn {  
  from {  
    opacity: 0;  
    transform: translateY(20px);  
  }  
  to {  
    opacity: 1;  
    transform: translateY(0);  
  }  
}
```

3.1. Временная функция для ключевых кадров

Пять ключевых кадров указаны для анимации с именем "bounce". Между первым и вторым ключевым кадром (то есть между 0% и 25%) используется функция замедления. Между вторым и третьим (то есть между 25% и 50%) — функция плавного ускорения. И так далее. Элемент будет перемещаться вверх по странице на 50px, замедляясь по мере того, как он достигнет своей наивысшей точки, а затем ускоряться, когда он упадет до 100px. Вторая половина анимации ведет себя аналогичным образом, но только перемещает элемент на 25px вверх по странице.

Временная функция, указанная в ключевом кадре to или 100%, игнорируется.



0%

Старт
низ-слева

25%

Прыжок
верх-центр

50%

Приземление
низ-центр

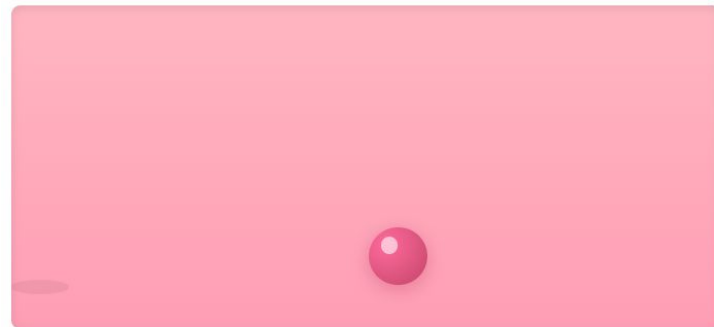
75%

Полупрыжок
середина

100%

Финиш
низ-справа

|| Пауза



0%

Старт
низ-слева

25%

Прыжок
верх-центр

50%

Приземление
низ-центр

75%

Полупрыжок
середина

100%

Финиш
низ-справа

|| Пауза

3.2. Название анимации: свойство `animation-name`

Свойство `animation-name` определяет список применяемых к элементу анимаций. Каждое имя используется для выбора ключевого кадра в правиле, которое предоставляет значения свойств для анимации. Если имя не соответствует ни одному ключевому кадру в правиле, нет свойств для анимации, отсутствует имя анимации, анимация не будет выполняться.

Если несколько анимаций пытаются изменить одно и то же свойство, то выполнится анимация, ближайшая к концу списка имен.

Имя анимации чувствительно к регистру, не допускается использование ключевого слова `none`. Свойство не наследуется.

3.2. Название анимации: свойство `animation-name`

<code>animation-name</code>	
Значения:	Описание
<code>none</code>	Означает отсутствие анимации. Также используется, чтобы отменить анимацию элемента из группы элементов, для которых задана анимация. Значение по умолчанию.
имя анимации	Имя анимации, которое связывает правило <code>@keyframes</code> с селектором.
<code>initial</code>	Устанавливает значение свойства в значение по умолчанию.
<code>inherit</code>	Наследует значение свойства от родительского элемента.

3.2. Название анимации: свойство `animation-name`

```
animation-name: none;  
animation-name: test-01;  
animation-name: -sliding;  
animation-name:  
moving-vertically;  
animation-name: test2;  
animation-name: test3, move4;  
animation-name: initial;  
animation-name: inherit;
```

3.3. Продолжительность анимации: свойство `animation-duration`

Свойство `animation-duration` определяет продолжительность одного цикла анимации. Задаётся в секундах `s` или миллисекундах `ms`. Если для элемента задано более одной анимации, то можно установить разное время для каждой, перечислив значения через запятую. Свойство не наследуется.

Значения:	Описание
время	Указывает время, которое анимация занимает для завершения одного цикла. Отрицательные значения недействительны. Если время равно <code>0s</code> , ключевые кадры анимации не действуют, но сама анимация происходит мгновенно. Значение по умолчанию <code>0s</code> .

3.3. Продолжительность анимации: свойство `animation-duration`

`animation-duration: .5s;`

`animation-duration: 200ms;`

`animation-duration: 2s, 10s;`

`animation-duration: 15s, 30s, 200ms;`

3.4. Временная функция: свойство `animation-timing-function`

Свойство `animation-timing-function` описывает, как будет развиваться анимация между каждой парой ключевых кадров. Во время задержки анимации временные функции не применяются.

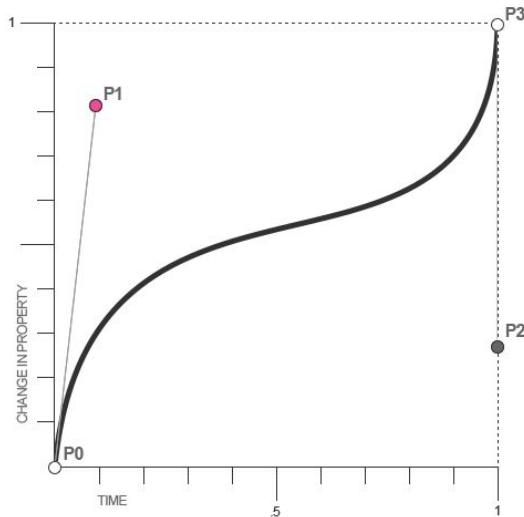
Свойство не наследуется.

animation-timing-function

Значения:	Описание
linear	Линейная функция, анимация происходит равномерно на протяжении всего времени, без колебаний в скорости.
Функции Безье	
ease	Функция по умолчанию, анимация начинается медленно, разгоняется быстро и замедляется в конце. Соответствует cubic-bezier(0.25,0.1,0.25,1).
ease-in	Анимация начинается медленно, а затем плавно ускоряется в конце. Соответствует cubic-bezier(0.42,0,1,1).
ease-out	Анимация начинается быстро и плавно замедляется в конце. Соответствует cubic-bezier(0,0,0.58,1).
ease-in-out	Анимация медленно начинается и медленно заканчивается. Соответствует cubic-bezier(0.42,0,0.58,1).

cubic-bezier
(x1, y1, x2,
y2)

Позволяет установить значения вручную.



P0 и P3 всегда находятся в одном месте, P0 — в точке (0,0), а P3 — в точке (1,1). По оси X показано время анимации, а по оси Y — изменяемое свойство. Кривая будет растягиваться по-разному в зависимости от того, какие значения вы дадите точкам P1 и P2 $(-\infty, \infty)$.

Пошаговые функции

step-start	Задаёт пошаговую анимацию, разбивая анимацию на отрезки, изменения происходят в начале каждого шага. Вычисляется в <code>steps(1, start)</code> .
step-end	Пошаговая анимация, изменения происходят в конце каждого шага. Вычисляется в <code>steps(1, end)</code> .
steps (количество шагов, положение шага)	Ступенчатая временная функция, которая принимает два параметра. Первый параметр указывает количество интервалов в функции. Это должно быть положительное целое число больше 0, если вторым параметром не является <code>jump-none</code> — в этом случае оно должно быть положительным целым числом больше 1. Второй параметр, который является необязательным, указывает позицию шага — момент, в котором начинается анимация, используя одно из следующих значений: ○ <code>jump-start</code> — первый шаг происходит при значении 0 ○ <code>jump-end</code> — последний шаг происходит при значении 1 ○ <code>jump-none</code> — все шаги происходят в пределах диапазона (0, 1) ○ <code>jump-both</code> — первый шаг происходит при значении 0, последний — при значении 1 ○ <code>start</code> — ведёт себя как <code>jump-start</code> ○ <code>end</code> — ведёт себя как <code>jump-end</code> Со значением <code>start</code> анимация начинается в начале каждого шага, со значением <code>end</code> — в конце каждого шага с задержкой. Задержка вычисляется как результат деления времени анимации на количество шагов. Если второй параметр не указан, используется значение по умолчанию <code>end</code>.

animation-timing-function

initial	Устанавливает значение свойства в значение по умолчанию.
inherit	Наследует значение свойства от родительского элемента.

`animation-timing-function: ease;`
`animation-timing-function: ease-in;`
`animation-timing-function: ease-out;`
`animation-timing-function: ease-in-out;`
`animation-timing-function: linear;`
`animation-timing-function: step-start;`
`animation-timing-function: step-end;`
`animation-timing-function:`
`cubic-bezier(0.1, 0.7, 1.0, 0.1);`
`animation-timing-function: steps(4, end);`
`animation-timing-function: ease, step-start,`
`cubic-bezier(0.1, 0.7, 1.0, 0.1);`
`animation-timing-function: initial;`
`animation-timing-function: inherit;`

CSS Demo: animation-timing-function

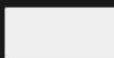
Reset

animation-timing-function: linear;

animation-timing-function: ease-in

animation-timing-function: steps(5)

animation-timing-function: cubic-b



CSS Demo: animation-timing-function

Reset

animation-timing-function: linear;

animation-timing-function: ease-in

animation-timing-function: steps(5)

animation-timing-function: cubic-b



Play

3.5. Повтор анимации: свойство `animation-iteration-count`

Свойство `animation-iteration-count` указывает, сколько раз проигрывается цикл анимации. Начальное значение 1 означает, что анимация будет воспроизводиться от начала до конца один раз. Это свойство часто используется в сочетании со значением `alternate` свойства `animation-direction`, которое заставляет анимацию воспроизводиться в обратном порядке в альтернативных циклах. Свойство не наследуется.

```
animation-iteration-count: infinite;  
animation-iteration-count: 3;  
animation-iteration-count: 2.5;  
animation-iteration-count: 2, 0, infinite;
```

3.5. Повтор анимации: свойство `animation-iteration-count`

<code>animation-iteration-count</code>	
Значения:	Описание
<code>infinite</code>	Анимация проигрывается бесконечно.
число	Анимация будет повторяться указанное количество раз. Если число не является целым числом, анимация закончится в середине последнего цикла. Отрицательные числа недействительны. Значение 0 вызывает мгновенное срабатывание анимации.

3.6. Направление анимации: свойство `animation-direction`

Свойство `animation-direction` определяет, должна ли анимация воспроизводиться в обратном порядке в некоторых или во всех циклах. Когда анимация воспроизводится в обратном порядке, временные функции также меняются местами. Например, при воспроизведении в обратном порядке функция `ease-in` будет вести себя как `ease-out`. Свойство не наследуется.

```
animation-direction: normal;
```

```
animation-direction: reverse;
```

```
animation-direction: alternate-reverse;
```

```
animation-direction: alternate, reverse,  
normal;
```

animation-direction

Значения:	Описание
normal	Все повторы анимации воспроизводятся так, как указано. Значение по умолчанию.
reverse	Все повторы анимации воспроизводятся в обратном направлении от того, как они были определены.
alternate	Каждый нечетный повтор цикла анимации воспроизводятся в нормальном направлении, каждый четный повтор воспроизводится в обратном направлении.
alternate-reverse	Каждый нечетный повтор цикла анимации воспроизводятся в обратном направлении, каждый четный повтор воспроизводится в нормальном направлении.
initial	Устанавливает значение свойства в значение по умолчанию.
inherit	Наследует значение свойства от родительского элемента.

3.7. Проигрывание анимации: свойство `animation-play-state`

Свойство `animation-play-state` определяет, будет ли анимация запущена или приостановлена. Остановка анимации внутри цикла возможна через использование этого свойства в скрипте JavaScript. Также можно останавливать анимацию при наведении курсора мыши на объект — состояние `:hover`.

Свойство не наследуется.

```
animation-play-state: running;
```

```
animation-play-state: paused;
```

```
animation-play-state: paused,  
running, running;
```

```
animation-play-state: initial;
```

```
animation-play-state: inherit;
```

animation-play-state

Значения:	Описание
running	Анимация выполняется. Значение по умолчанию.
paused	Анимация приостанавливается. При перезапуске анимация начинается с того места, где она была остановлена, как если бы «часы», управляющие анимацией, остановились и снова запустились. Если анимация остановлена во время задержки, при повторном воспроизведении время задержки также возобновляется.
initial	Устанавливает значение свойства в значение по умолчанию.
inherit	Наследует значение свойства от родительского элемента.

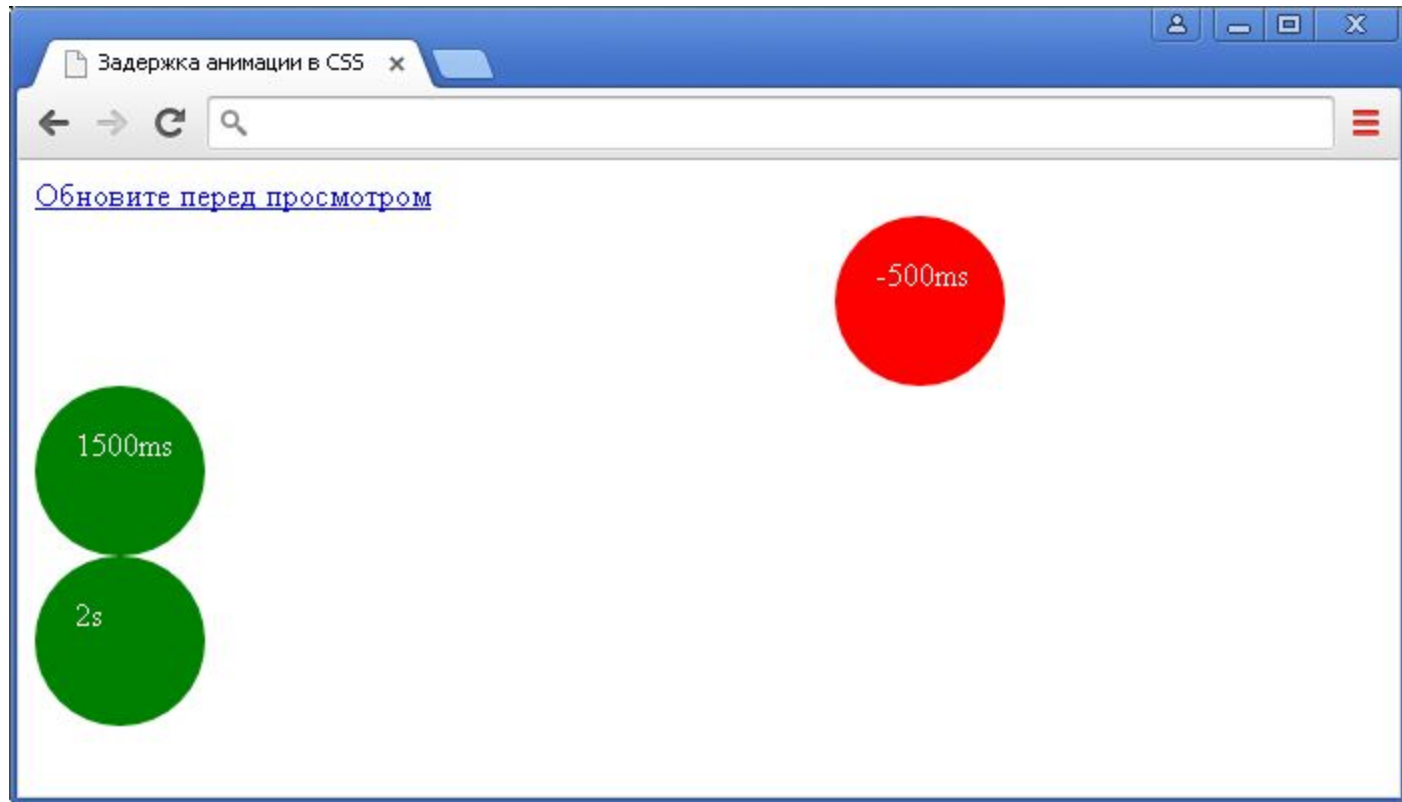
3.8. Задержка анимации: свойство `animation-delay`

Свойство `animation-delay` определяет, когда анимация начнется. Задается в секундах `s` или миллисекундах `ms`. Свойство не наследуется.

```
animation-delay: 5s;
```

```
animation-delay: 3s, 10ms;
```

время	Время определяет длительность задержки между началом анимации (когда анимация применяется к элементу через свойства) и когда она начинает выполняться. Отрицательные значения разрешены, такая задержка начинает анимацию с определенного момента внутри её цикла, т.е. со времени, указанного в задержке. Это позволяет применять анимацию к нескольким элементам со сдвигом фазы, изменяя лишь время задержки. Чтобы анимация началась с середины, нужно задать отрицательную задержку, равную половине времени, установленному в <code>animation-duration</code> . Значение по умолчанию <code>0s</code> .
-------	---



3.9. Состояние элемента до и после воспроизведения анимации: свойство `animation-fill-mode`

Свойство `animation-fill-mode` определяет, какие значения применяются анимацией вне времени ее выполнения. Когда анимация завершается, элемент возвращается к своим исходным стилям. По умолчанию анимация не влияет на значения свойств `animation-name` и `animation-delay`, когда анимация применяется к элементу. Кроме того, по умолчанию анимация не влияет на значения свойств `animation-duration` и `animation-iteration-count` после ее завершения. Свойство `animation-fill-mode` может переопределить это поведение.

Свойство не наследуется.

animation-fill-mode

Значения:	Описание
none	Значение по умолчанию. Состояние элемента не меняется до или после воспроизведения анимации.
forwards	После того, как анимация заканчивается (как определено значением animation-iteration-count), анимация будет применять значения свойств к моменту окончания анимации. Если animation-iteration-count больше нуля, применяются значения для конца последней завершенной итерации анимации (а не значения для начала итерации, которое будет следующим). Если значение animation-iteration-count равно нулю, применяемыми значениями будут те, которые начнут первую итерацию (так же, как и в режиме animation-fill-mode: backwards;).
backwards	В течение периода, определенного с помощью animation-delay, анимация будет применять значения свойств, определенные в ключевом кадре, которые начнут первую итерацию анимации. Это либо значения ключевого кадра from (когда animation-direction: normal или animation-direction: alternate), либо значения ключевого кадра to (когда animation-direction: reverse или animation-direction: alternate).
both	Позволяет оставлять элемент в первом ключевом кадре до начала анимации (игнорируя положительное значение задержки) и задерживать на последнем кадре до конца последней анимации.

3.9. Состояние элемента до и после воспроизведения анимации: свойство `animation-fill-mode`

`animation-fill-mode`: none;

`animation-fill-mode`: forwards;

`animation-fill-mode`: backwards;

`animation-fill-mode`: both;

`animation-fill-mode`: none, backwards;

`animation-fill-mode`: both, forwards, none;

3.10. Краткая запись анимации: свойство `animation`

Все параметры воспроизведения анимации можно объединить в одном свойстве — `animation`, перечислив их через пробел:
`animation: animation-name animation-duration animation-timing-function animation-delay animation-iteration-count animation-direction;`

Для воспроизведения анимации достаточно указать только два свойства — `animation-name` и `animation-duration`, остальные свойства примут значения по умолчанию. Порядок перечисления свойств не имеет значения, единственное, время выполнения анимации `animation-duration` обязательно должно стоять перед задержкой `animation-delay`.

/* Полная запись */

```
.element {  
  animation-name: slideIn;  
  animation-duration: 1s;  
  animation-timing-function: ease-out;  
  animation-delay: 0.3s;  
  animation-iteration-count: infinite;  
  animation-direction: alternate;  
  animation-fill-mode: forwards;  
}
```

/* Краткая запись */

```
.element {  
  animation: slideIn 1s ease-out 0.3s infinite  
    alternate forwards;  
}
```

Порядок в краткой записи: animation-name, animation-duration, animation-timing-function, animation-delay, animation-iteration-count, animation-direction, animation-fill-mode

3.11. Множественные анимации

Для одного элемента можно задавать несколько анимаций, перечислив их названия через запятую:

```
div {animation: shadow 1s ease-in-out 0.5s alternate, move 5s linear 2s;}
```

Заключение

CSS-трансформации и анимации — это инструменты для создания интерактивных визуальных эффектов без JavaScript: `transform` изменяет вид элемента (перемещение, поворот, масштаб, наклон) без влияния на поток документа, `transition` обеспечивает плавные переходы между двумя состояниями при событиях (hover, click), а `animation` воспроизводит сложные многокадровые сценарии с автостартом и зацикливанием.